# NetApp Astra Trident の概要

NetApp Solutions

NetApp
April 07, 2022

This PDF was generated from https://docs.netapp.com/ja-jp/netapp-solutions/containers/rh-os-n_trident_ontap_nfs.html on April 07, 2022. Always check docs.netapp.com for the latest.

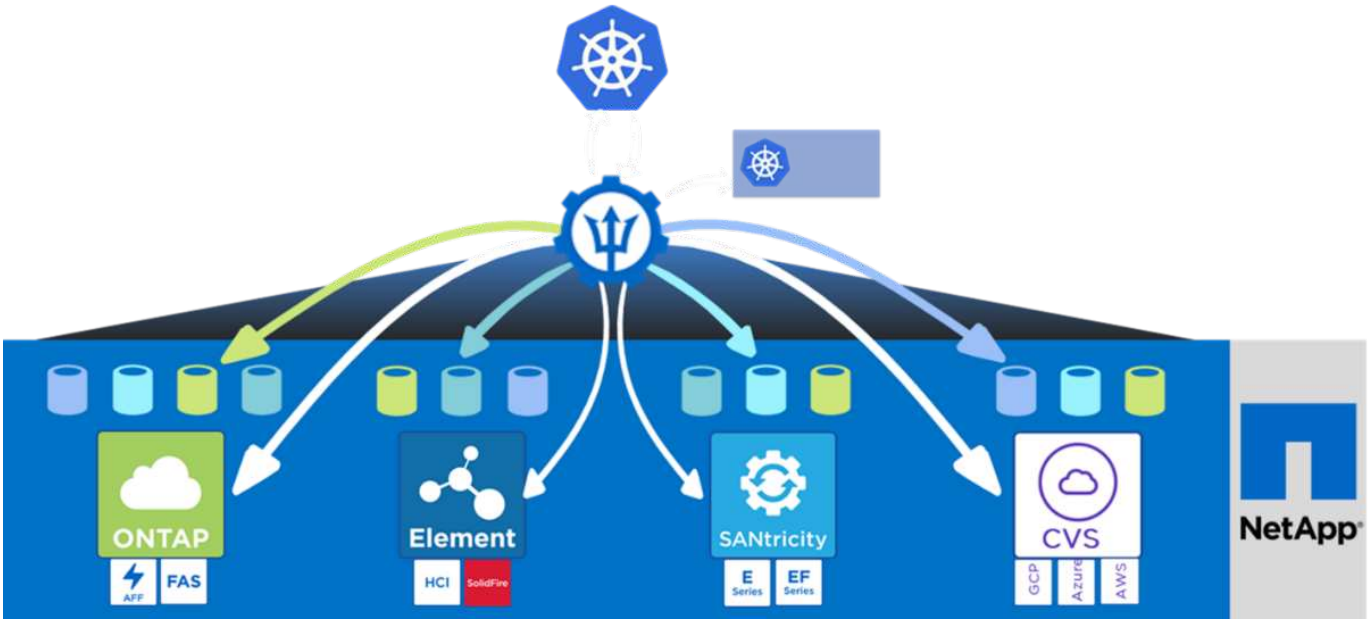
# 目次

Astra Trident の概要 . . . . . 1
Astra Trident をダウンロード . . . . . 1
Helm を使用して Trident Operator をインストールします . . . . . 3
Trident Operator を手動でインストールします . . . . . 5
ワーカーノードをストレージ用に準備する . . . . . 8
ストレージシステムバックエンドを作成 . . . . . 13
NetApp ONTAP の NFS 構成 . . . . . 13
NetApp ONTAP iSCSI 構成 . . . . . 15
NetApp Element iSCSI 構成 . . . . . 18

# Astra Trident の概要

Astra Trident は、コンテナや Kubernetes ディストリビューション向けの、 Red Hat OpenShift などのオープンソースで完全にサポートされているストレージオーケストレーションツールです。Trident は、 NetApp ONTAP や Element ストレージシステムを含むネットアップストレージポートフォリオ全体と連携し、 NFS 接続と iSCSI 接続もサポートします。Trident を使用すると、ストレージ管理者の手を煩わせることなく、エンドユーザがネットアップストレージシステムからストレージをプロビジョニングして管理できるため、DevOps ワークフローが高速化されます。

管理者は、プロジェクトのニーズやストレージシステムモデルに基づいて複数のストレージバックエンドを構成し、圧縮、特定のディスクタイプ、 QoS レベルなどの高度なストレージ機能を有効にして一定のレベルのパフォーマンスを保証できます。定義されたバックエンドは、プロジェクトの開発者が永続的ボリューム要求（ PVC ）を作成し、永続的ストレージをオンデマンドでコンテナに接続するために使用できます。




Astra Trident は、 Kubernetes と同様、迅速な開発サイクルを 1 年に 4 回リリースしています。

2022 年 1 月にリリースされた最新バージョンの Astra Trident は 22.01 です。Trident のどのバージョンがサポートされているかを確認できます Kubernetes ディストリビューションのテストに使用 "こちらをご覧ください"。

20.04 リリース以降、 Trident のセットアップは Trident オペレータによって実行されます。オペレータが大規模な導入を容易にし、 Trident インストールの一部として導入されたポッドの自己修復などの追加サポートを提供します。

21.01 リリースでは、 Trident Operator のインストールを容易にするために Helm チャートを使用できるようになりました。

# Astra Trident をダウンロード

導入したユーザクラスタに Trident をインストールし、永続ボリュームをプロビジョニングするには、次の手順を実行します。

1. インストールアーカイブを管理ワークステーションにダウンロードし、内容を展開します。Trident の最新

バージョンは 22.01 で、ダウンロードできます "こちらをご覧ください"。

```
[netapp-user@rhel7 ~]$ wget
https://github.com/NetApp/trident/releases/download/v22.01.0/trident-
installer-22.01.0.tar.gz
--2021-05-06 15:17:30--
https://github.com/NetApp/trident/releases/download/v22.01.0/trident-
installer-22.01.0.tar.gz
Resolving github.com (github.com)... 140.82.114.3
Connecting to github.com (github.com)|140.82.114.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github-
releases.githubusercontent.com/77179634/a4fa9f00-a9f2-11eb-9053-
98e8e573d4ae?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-
Credential=AKIAIWNJYAX4CSVEH53A%2F20210506%2Fus-east-
1%2Fs3%2Faws4_request&X-Amz-Date=20210506T191643Z&X-Amz-Expires=300&X-
Amz-
Signature=8a49a2a1e08c147d1ddd8149ce45a5714f9853fee19bb1c507989b9543eb36
30&X-Amz-
SignedHeaders=host&actor_id=0&key_id=0&repo_id=77179634&response-
content-disposition=attachment%3B%20filename%3Dtrident-installer-
22.01.0.tar.gz&response-content-type=application%2Foctet-stream
[following]
--2021-05-06 15:17:30--  https://github-
releases.githubusercontent.com/77179634/a4fa9f00-a9f2-11eb-9053-
98e8e573d4ae?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-
Credential=AKIAIWNJYAX4CSVEH53A%2F20210506%2Fus-east-
1%2Fs3%2Faws4_request&X-Amz-Date=20210506T191643Z&X-Amz-Expires=300&X-
Amz-
Signature=8a49a2a1e08c147d1ddd8149ce45a5714f9853fee19bb1c507989b9543eb36
30&X-Amz-
SignedHeaders=host&actor_id=0&key_id=0&repo_id=77179634&response-
content-disposition=attachment%3B%20filename%3Dtrident-installer-
22.01.0.tar.gz&response-content-type=application%2Foctet-stream
Resolving github-releases.githubusercontent.com (github-
releases.githubusercontent.com)... 185.199.108.154, 185.199.109.154,
185.199.110.154, ...
Connecting to github-releases.githubusercontent.com (github-
releases.githubusercontent.com)|185.199.108.154|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 38349341 (37M) [application/octet-stream]
Saving to: 'trident-installer-22.01.0.tar.gz'

100%[=================================================================
==============================================>] 38,349,341  88.5MB/s
in 0.4s
```

```
2021-05-06 15:17:30 (88.5 MB/s) - 'trident-installer-22.01.0.tar.gz'
saved [38349341/38349341]
```

2. ダウンロードしたバンドルから Trident のインストールを解凍します。

```
[netapp-user@rhel7 ~]$ tar -xzf trident-installer-22.01.0.tar.gz
[netapp-user@rhel7 ~]$ cd trident-installer/
[netapp-user@rhel7 trident-installer]$
```

## Helm を使用して Trident Operator をインストールします

1. Trident にはこのファイルを渡すオプションがないため、まず、ユーザクラスタの「 kubeconfig 」ファイルの場所を環境変数として設定します。

```
[netapp-user@rhel7 trident-installer]$ export KUBECONFIG=~/ocp-
install/auth/kubeconfig
```

2. Helm コマンドを実行し、ユーザークラスタに trident 名前空間を作成しながら、 Helball ディレクトリ内の tarball から Trident 演算子をインストールします。

```
[netapp-user@rhel7 trident-installer]$ helm install trident
helm/trident-operator-22.01.0.tgz --create-namespace --namespace trident
NAME: trident
LAST DEPLOYED: Fri May  7 12:54:25 2021
NAMESPACE: trident
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Thank you for installing trident-operator, which will deploy and manage
NetApp's Trident CSI
storage provisioner for Kubernetes.

Your release is named 'trident' and is installed into the 'trident'
namespace.
Please note that there must be only one instance of Trident (and
trident-operator) in a Kubernetes cluster.

To configure Trident to manage storage resources, you will need a copy
of tridentctl, which is
available in pre-packaged Trident releases.  You may find all Trident
releases and source code
online at https://github.com/NetApp/trident.

To learn more about the release, try:

  $ helm status trident
  $ helm get all trident
```

3. Trident が正しくインストールされていることを確認するには、ネームスペースで実行されているポッドを確認するか、 tridentctl バイナリを使用してインストールされているバージョンを確認します。

```
[netapp-user@rhel7 trident-installer]$ oc get pods -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-5z45l                   1/2     Running   2          30s
trident-csi-696b685cf8-htdb2        6/6     Running   0          30s
trident-csi-b74p2                   2/2     Running   0          30s
trident-csi-lrw4n                   2/2     Running   0          30s
trident-operator-7c748d957-gr2gw    1/1     Running   0          36s

[netapp-user@rhel7 trident-installer]$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 22.01.0          | 22.01.0          |
+----------------+----------------+
```

場合によっては、お客様の環境で Trident の導入のカスタマイズが必要になることもあります。このような場合は、 Trident のオペレータを手動でインストールし、含まれているマニフェストを更新して配置をカスタマイズすることもできます。

## Trident Operator を手動でインストールします

1. Trident にはこのファイルを渡すオプションがないため、まず、ユーザクラスタの「 kubeconfig 」ファイルの場所を環境変数として設定します。

```
[netapp-user@rhel7 trident-installer]$ export KUBECONFIG=~/ocp-
install/auth/kubeconfig
```

2. 'trident-installer' ディレクトリには ' 必要なすべてのリソースを定義するマニフェストが含まれています適切なマニフェストを使用して、「 TridentOrchestrator 」カスタムリソース定義を作成します。

```
[netapp-user@rhel7 trident-installer]$ oc create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
customresourcedefinition.apiextensions.k8s.io/tridentorchestrators.tride
nt.netapp.io created
```

3. 存在しない場合は、指定されたマニフェストを使用して、クラスタ内に Trident ネームスペースを作成します。

```
[netapp-user@rhel7 trident-installer]$ oc apply -f deploy/namespace.yaml
namespace/trident created
```

4. トライデントオペレータの配備に必要なリソースを作成しますたとえば ' オペレータ用のサービスアカウント 'ClusterRole' および 'ClusterRoleBind' を 'ServiceAccount' 専用の 'PodSecurityPolicy' またはオペレータ自体に割り当てます

```
[netapp-user@rhel7 trident-installer]$ oc create -f deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created
```

5. 次のコマンドを使用すると、展開後にオペレータのステータスを確認できます。

```
[netapp-user@rhel7 trident-installer]$ oc get deployment -n trident
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
trident-operator   1/1     1            1           23s
[netapp-user@rhel7 trident-installer]$ oc get pods -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-operator-66f48895cc-lzczk   1/1     Running   0          41s
```

6. オペレータが導入したら、 Trident をインストールできます。これには 'TridentOrchestrator を作成する必要があります

```
[netapp-user@rhel7 trident-installer]$ oc create -f
deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created
[netapp-user@rhel7 trident-installer]$ oc describe torc trident
Name:         trident
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  trident.netapp.io/v1
Kind:         TridentOrchestrator
Metadata:
  Creation Timestamp:  2021-05-07T17:00:28Z
  Generation:          1
  Managed Fields:
    API Version:  trident.netapp.io/v1
    Fields Type:  FieldsV1
    fieldsV1:
      f:spec:
        .:
        f:debug:
        f:namespace:
```

```
    Manager:      kubectl-create
    Operation:    Update
    Time:         2021-05-07T17:00:28Z
    API Version:  trident.netapp.io/v1
    Fields Type:  FieldsV1
    fieldsV1:
      f:status:
        .:
        f:currentInstallationParams:
          .:
          f:IPv6:
          f:autosupportHostname:
          f:autosupportImage:
          f:autosupportProxy:
          f:autosupportSerialNumber:
          f:debug:
          f:enableNodePrep:
          f:imagePullSecrets:
          f:imageRegistry:
          f:k8sTimeout:
          f:kubeletDir:
          f:logFormat:
          f:silenceAutosupport:
          f:tridentImage:
        f:message:
        f:namespace:
        f:status:
        f:version:
    Manager:         trident-operator
    Operation:       Update
    Time:            2021-05-07T17:00:28Z
  Resource Version:  931421
  Self Link:
/apis/trident.netapp.io/v1/tridentorchestrators/trident
  UID:               8a26a7a6-dde8-4d55-9b66-a7126754d81f
Spec:
  Debug:      true
  Namespace:  trident
Status:
  Current Installation Params:
    IPv6:                       false
    Autosupport Hostname:
    Autosupport Image:          netapp/trident-autosupport:21.01
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:                      true
```

```
    Enable Node Prep:           false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:22.01.0
  Message:                Trident installed
  Namespace:              trident
  Status:                 Installed
  Version:                v22.01.0
Events:
  Type    Reason      Age   From                        Message
  ----    ------      ----  ----                        -------
  Normal  Installing  80s   trident-operator.netapp.io  Installing
Trident
  Normal  Installed   68s   trident-operator.netapp.io  Trident
installed
```

7. Trident が正しくインストールされていることを確認するには、ネームスペースで実行されているポッドを確認するか、 tridentctl バイナリを使用してインストールされているバージョンを確認します。

```
[netapp-user@rhel7 trident-installer]$ oc get pods -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-bb64c6cb4-lmd6h         6/6     Running   0          82s
trident-csi-gn59q                   2/2     Running   0          82s
trident-csi-m4szj                   2/2     Running   0          82s
trident-csi-sb9k9                   2/2     Running   0          82s
trident-operator-66f48895cc-lzczk   1/1     Running   0          2m39s

[netapp-user@rhel7 trident-installer]$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 22.01.0          | 22.01.0          |
+----------------+----------------+
```

# ワーカーノードをストレージ用に準備する

## NFS

ほとんどの Kubernetes ディストリビューションには、 Red Hat OpenShift など、デフォルトでインストールされる NFS バックエンドをマウントするパッケージとユーティリティが付属しています。

ただし NFSv3 については、クライアントとサーバ間の同時処理をネゴシエートするメカニズムはありません。したがって ' サーバが接続のウィンドウ・サイズを小さくしなくても NFS 接続の最適なパフォーマンスを確保できるように ' クライアント側 sunrpc スロット・テーブル・エントリーの最大数は ' サーバ上でサポートされている値と手動で同期する必要があります

ONTAP でサポートされる sunrpcslot table エントリの最大数は 128 です。つまり、 ONTAP は、一度に 128 個の NFS 要求を同時に処理できます。ただし、 Red Hat CoreOS / Red Hat Enterprise Linux では、接続ごとに最大 65 、 536 の sunrpc スロットテーブルエントリがデフォルトでサポートされます。この値を 128 に設定する必要があります。これは OpenShift のマシン構成オペレータ（ MCO ）を使用して実行できます。

OpenShift ワーカーノードで最大 sunrpc スロットテーブルエントリを変更するには、次の手順を実行します。

1. OCP Web コンソールにログインし、 [ 計算 ]>[ マシン構成 ] に移動します。[ マシン構成の作成 ] をクリックします。YAML ファイルをコピーして貼り付け、 [ 作成 ] をクリックします。

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: 98-worker-nfs-rpc-slot-tables
  labels:
    machineconfiguration.openshift.io/role: worker
spec:
  config:
    ignition:
      version: 3.2.0
    storage:
      files:
        - contents:
            source: data:text/plain;charset=utf-
8; base64,b3B0aW9ucyBzdW5ycGMgdGNwX21heF9zbG90X3RhYmxlX2VudHJpZXM9MTI4Cg=
=
          filesystem: root
          mode: 420
          path: /etc/modprobe.d/sunrpc.conf
```

2. MCO が作成されたら、すべてのワーカーノードに設定を適用し、 1 つずつ再起動する必要があります。プロセス全体には約 20~30 分かかります。「 OC GET MCP 」を使用してマシン構成が適用されているかどうかを確認し、ワーカーのマシン構成プールが更新されていることを確認します。

```
[netapp-user@rhel7 openshift-deploy]$ oc get mcp
NAME     CONFIG                                     UPDATED   UPDATING
DEGRADED
master   rendered-master-a520ae930e1d135e0dee7168   True      False
False
worker   rendered-worker-de321b36eeba62df41feb7bc   True      False
False
```

## iSCSI

iSCSI プロトコルによるブロックストレージボリュームのマッピングを許可するようにワーカーノードを準備するには、その機能をサポートするために必要なパッケージをインストールする必要があります。

Red Hat OpenShift では、 MCO （マシン構成オペレータ）を展開後にクラスタに適用することによって処理されます。

ワーカーノードで iSCSI サービスを実行するように設定するには、次の手順を実行します。

1. OCP Web コンソールにログインし、 [ 計算 ]>[ マシン構成 ] に移動します。[ マシン構成の作成 ] をクリックします。YAML ファイルをコピーして貼り付け、 [ 作成 ] をクリックします。

   マルチパスを使用しない場合：

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 99-worker-element-iscsi
spec:
  config:
    ignition:
      version: 3.2.0
    systemd:
      units:
        - name: iscsid.service
          enabled: true
          state: started
  osImageURL: ""
```

   マルチパスを使用する場合：

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: 99-worker-ontap-iscsi
  labels:
    machineconfiguration.openshift.io/role: worker
spec:
  config:
    ignition:
      version: 3.2.0
    storage:
      files:
      - contents:
          source: data:text/plain;charset=utf-
8; base64,ZGVmYXVsdHMgewogICAgICAgIHVzZXJfZnJpZW5kbHlfbmFtZXMgbm8KICAgICA
gICBmaW5kX211bHRpcGF0aHMgbm8KfQoKYmxhY2tsaXN0X2V4Y2VwdGlvbnMgewogICAgICA
gIHByb3BlcnR5ICIoU0NTSV9JREVOVF98SURfV1dOKSIKfQoKYmxhY2tsaXN0IHsKfQoK
          verification: {}
        filesystem: root
        mode: 400
        path: /etc/multipath.conf
    systemd:
      units:
        - name: iscsid.service
          enabled: true
          state: started
        - name: multipathd.service
          enabled: true
          state: started
  osImageURL: ""
```

2. 構成の作成後、約 20~30 分で設定がワーカーノードに適用され、再ロードされます。「 OC GET MCP 」を使用してマシン構成が適用されているかどうかを確認し、ワーカーのマシン構成プールが更新されていることを確認します。ワーカーノードにログインして、 iscsid サービスが実行されている（マルチパスを使用している場合、 multipathd サービスが実行されている）ことを確認することもできます。

```
[netapp-user@rhel7 openshift-deploy]$ oc get mcp
NAME     CONFIG                                     UPDATED   UPDATING
DEGRADED
master   rendered-master-a520ae930e1d135e0dee7168   True      False
False
worker   rendered-worker-de321b36eeba62df41feb7bc   True      False
False

[netapp-user@rhel7 openshift-deploy]$ ssh core@10.61.181.22 sudo
systemctl status iscsid
● iscsid.service - Open-iSCSI
   Loaded: loaded (/usr/lib/systemd/system/iscsid.service; enabled;
vendor preset: disabled)
   Active: active (running) since Tue 2021-05-26 13:36:22 UTC; 3 min ago
     Docs: man:iscsid(8)
           man:iscsiadm(8)
 Main PID: 1242 (iscsid)
   Status: "Ready to process requests"
    Tasks: 1
   Memory: 4.9M
      CPU: 9ms
   CGroup: /system.slice/iscsid.service
           └─1242 /usr/sbin/iscsid -f

[netapp-user@rhel7 openshift-deploy]$ ssh core@10.61.181.22 sudo
systemctl status multipathd
 ● multipathd.service - Device-Mapper Multipath Device Controller
   Loaded: loaded (/usr/lib/systemd/system/multipathd.service; enabled;
vendor preset: enabled)
   Active: active (running) since Tue 2021-05-26 13:36:22 UTC; 3 min ago
  Main PID: 918 (multipathd)
    Status: "up"
    Tasks: 7
    Memory: 13.7M
    CPU: 57ms
    CGroup: /system.slice/multipathd.service
            └─918 /sbin/multipathd -d -s
```

また、適切なフラグを指定して「 OC debug 」コマンドを実行することにより、MachineConfig が正常に適用され、サービスが正常に開始されたことを確認することもできます。

# ストレージシステムバックエンドを作成

Astra Trident Operator のインストールが完了したら、使用するネットアップストレージプラットフォームに合わせてバックエンドを設定する必要があります。Astra Trident のセットアップと設定を続行するには、次のリンクを参照してください。

- "NetApp ONTAP NFS"
- "NetApp ONTAP iSCSI の略"
- "NetApp Element iSCSI の略"

"次：解決策の検証 / ユースケース：ネットアップを使用した Red Hat OpenShift 。"

## NetApp ONTAP の NFS 構成

Trident を NetApp ONTAP ストレージシステムと統合するには、ストレージシステムとの通信を可能にするバックエンドを作成する必要があります。

1. ダウンロードしたインストールアーカイブのサンプルバックエンドファイルは、「 sample -input 」フォルダ階層にあります。NFS を提供している NetApp ONTAP システムの場合は、「 backend-ontap/nas.json 」ファイルを作業ディレクトリにコピーし、ファイルを編集します。

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/backends-
samples/ontap-nas/backend-ontap-nas.json ./
[netapp-user@rhel7 trident-installer]$ vi backend-ontap-nas.json
```

2. backendName 、 managementLIF 、 dataLIF 、 SVM 、ユーザ名を編集します。 パスワードの値を入力します。

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "ontap-nas+10.61.181.221",
    "managementLIF": "172.21.224.201",
    "dataLIF": "10.61.181.221",
    "svm": "trident_svm",
    "username": "cluster-admin",
    "password": "password"
}
```

> カスタムの backendName 値は、簡単に識別できるように NFS を提供するストレージ DriverName とデータ LIF を組み合わせて定義することを推奨します。

3. このバックエンドファイルを設定した状態で、次のコマンドを実行して最初のバックエンドを作成します。

```
[netapp-user@rhel7 trident-installer]$ ./tridentctl -n trident create
backend -f backend-ontap-nas.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas+10.61.181.221 | ontap-nas      | be7a619d-c81d-445c-b80c-
5c87a73c5b1e | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

4. バックエンドを作成したら、次にストレージクラスを作成する必要があります。バックエンドと同様に、sample_inputs フォルダにある環境用に編集可能なサンプルのストレージクラスファイルがあります。作業ディレクトリにコピーし、作成したバックエンドを反映するために必要な編集を行います。

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/storage-class-
samples/storage-class-csi.yaml.templ ./storage-class-basic.yaml
[netapp-user@rhel7 trident-installer]$ vi storage-class-basic.yaml
```

5. このファイルに対して行う必要がある唯一の編集は ' バックエンドタイプの値を ' 新しく作成されたバックエンドのストレージドライバの名前に定義することですまた、名前フィールドの値もメモしておきます。この値は、以降の手順で参照する必要があります。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-nas"
```

> このファイルに定義されているオプションのフィールド「 fsType 」があります。この行は NFS バックエンドで削除できます。

6. oc` コマンドを実行して ' ストレージ・クラスを作成します

```
[netapp-user@rhel7 trident-installer]$ oc create -f storage-class-
basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

7. ストレージクラスを作成したら、最初の永続的ボリューム要求（ PVC ）を作成する必要があります。sample_inputs にもあるこのアクションを実行するために使用できるサンプルの 'pvc-basicy.yaml ファイルがあります

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/pvc-samples/pvc-
basic.yaml ./
[netapp-user@rhel7 trident-installer]$ vi pvc-basic.yaml
```

8. このファイルに対して行う必要がある唯一の編集は ''torageClassName` フィールドが作成したものと一致することを確認することですプロビジョニングするワークロードによって必要に応じて、 PVC 定義をさらにカスタマイズできます。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

9. 「 OC 」コマンドを発行して、 PVC を作成します。作成中の元のボリュームのサイズによっては作成にしばらく時間がかかることがあるため、作成が完了した時点でこのプロセスを監視できます。

```
[netapp-user@rhel7 trident-installer]$ oc create -f pvc-basic.yaml
persistentvolumeclaim/basic created

[netapp-user@rhel7 trident-installer]$ oc get pvc
NAME     STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
basic    Bound    pvc-b4370d37-0fa4-4c17-bd86-94f96c94b42d   1Gi
RWO            basic-csi      7s
```

"次：解決策の検証 / ユースケース"

# NetApp ONTAP iSCSI 構成

Trident を NetApp ONTAP ストレージシステムと統合するには、ストレージシステムとの通信を可能にするバックエンドを作成する必要があります。

1. ダウンロードしたインストールアーカイブのサンプルバックエンドファイルは、「 sample -input 」フォ

ルダ階層にあります。iSCSI を提供している NetApp ONTAP システムの場合は、「 backend-ontap -san.json 」ファイルを作業ディレクトリにコピーし、ファイルを編集します。

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/backends-
samples/ontap-san/backend-ontap-san.json ./
[netapp-user@rhel7 trident-installer]$ vi backend-ontap-san.json
```

2. このファイルで管理 LIF 、データ LIF 、 SVM 、ユーザ名、パスワードの値を編集します。

```
{
  "version": 1,
  "storageDriverName": "ontap-san",
  "managementLIF": "172.21.224.201",
  "dataLIF": "10.61.181.240",
  "svm": "trident_svm",
  "username": "admin",
  "password": "password"
}
```

3. このバックエンドファイルを設定した状態で、次のコマンドを実行して最初のバックエンドを作成します。

```
[netapp-user@rhel7 trident-installer]$ ./tridentctl -n trident create
backend -f backend-ontap-san.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| ontapsan_10.61.181.241 | ontap-san      | 6788533c-7fea-4a35-b797-
fb9bb3322b91 | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

4. バックエンドを作成したら、次にストレージクラスを作成する必要があります。バックエンドと同様に、sample_inputs フォルダにある環境用に編集可能なサンプルのストレージクラスファイルがあります。作業ディレクトリにコピーし、作成したバックエンドを反映するために必要な編集を行います。

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/storage-class-
samples/storage-class-csi.yaml.templ ./storage-class-basic.yaml
[netapp-user@rhel7 trident-installer]$ vi storage-class-basic.yaml
```

5. このファイルに対して行う必要がある唯一の編集は ' バックエンドタイプの値を ' 新しく作成されたバックエンドのストレージドライバの名前に定義することですまた、名前フィールドの値もメモしておきます。この値は、以降の手順で参照する必要があります。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-san"
```

> このファイルに定義されているオプションのフィールド「 fsType 」があります。iSCSI バックエンドでは、この値を特定の Linux ファイルシステムタイプ（ XFS 、 ext4 など）に設定することも、 OpenShift が使用するファイルシステムを決定できるようにするために削除することもできます。

6. oc` コマンドを実行して ' ストレージ・クラスを作成します

```
[netapp-user@rhel7 trident-installer]$ oc create -f storage-class-
basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

7. ストレージクラスを作成したら、最初の永続的ボリューム要求（ PVC ）を作成する必要があります。sample_inputs にもあるこのアクションを実行するために使用できるサンプルの 'pvc-basicy.yaml ファイルがあります

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/pvc-samples/pvc-
basic.yaml ./
[netapp-user@rhel7 trident-installer]$ vi pvc-basic.yaml
```

8. このファイルに対して行う必要がある唯一の編集は ''torageClassName` フィールドが作成したものと一致することを確認することですプロビジョニングするワークロードによって必要に応じて、 PVC 定義をさらにカスタマイズできます。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

9. 「 OC 」コマンドを発行して、 PVC を作成します。作成中の元のボリュームのサイズによっては作成にしばらく時間がかかることがあるため、作成が完了した時点でこのプロセスを監視できます。

```
[netapp-user@rhel7 trident-installer]$ oc create -f pvc-basic.yaml
persistentvolumeclaim/basic created

[netapp-user@rhel7 trident-installer]$ oc get pvc
NAME    STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
basic   Bound    pvc-7ceac1ba-0189-43c7-8f98-094719f7956c   1Gi
RWO            basic-csi      3s
```

"次：解決策の検証 / ユースケース"

# NetApp Element iSCSI 構成

Trident を NetApp Element ストレージシステムと統合するには、 iSCSI プロトコルを使用してストレージシステムと通信できるバックエンドを作成する必要があります。

1. ダウンロードしたインストールアーカイブのサンプルバックエンドファイルは、「 sample -input 」フォルダ階層にあります。iSCSI を提供している NetApp Element システムの場合、「 backend-solidfire.json 」ファイルを作業ディレクトリにコピーし、ファイルを編集します。

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/backends-
samples/solidfire/backend-solidfire.json ./
[netapp-user@rhel7 trident-installer]$ vi ./backend-solidfire.json
```

   a. エンドポイント行のユーザ、パスワード、および MVIP 値を編集します。
   b. 「仮想 IP 」の値を編集します。

```
 {
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://trident:password@172.21.224.150/json-
rpc/8.0",
    "SVIP": "10.61.180.200:3260",
    "TenantName": "trident",
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS":
2000, "burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS":
6000, "burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS":
8000, "burstIOPS": 10000}}]
}
```

2. このバックエンドファイルを設定したら、次のコマンドを実行して最初のバックエンドを作成します。

```
[netapp-user@rhel7 trident-installer]$ ./tridentctl -n trident create
backend -f backend-solidfire.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| solidfire_10.61.180.200 | solidfire-san  | b90783ee-e0c9-49af-8d26-
3ea87ce2efdf | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

3. バックエンドを作成したら、次にストレージクラスを作成する必要があります。バックエンドと同様に、sample_inputs フォルダにある環境用に編集可能なサンプルのストレージクラスファイルがあります。作業ディレクトリにコピーし、作成したバックエンドを反映するために必要な編集を行います。

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/storage-class-
samples/storage-class-csi.yaml.templ ./storage-class-basic.yaml
[netapp-user@rhel7 trident-installer]$ vi storage-class-basic.yaml
```

4. このファイルに対して行う必要がある唯一の編集は ' バックエンドタイプの値を ' 新しく作成されたバックエンドのストレージドライバの名前に定義することですまた、名前フィールドの値もメモしておきます。この値は、以降の手順で参照する必要があります。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "solidfire-san"
```

このファイルに定義されているオプションのフィールド「 fsType 」があります。iSCSI バックエンドでは、この値を特定の Linux ファイルシステムタイプ（ XFS 、 ext4 など）に設定するか、 OpenShift で使用するファイルシステムを決定できるようにするためにこの値を削除できます。

5. oc` コマンドを実行して ' ストレージ・クラスを作成します

```
[netapp-user@rhel7 trident-installer]$ oc create -f storage-class-
basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

6. ストレージクラスを作成したら、最初の永続的ボリューム要求（ PVC ）を作成する必要があります。sample_inputs にもあるこのアクションを実行するために使用できるサンプルの 'pvc-basicy.yaml ファイルがあります

```
[netapp-user@rhel7 trident-installer]$ cp sample-input/pvc-samples/pvc-
basic.yaml ./
[netapp-user@rhel7 trident-installer]$ vi pvc-basic.yaml
```

7. このファイルに対して行う必要がある唯一の編集は ''torageClassName` フィールドが作成したものと一致することを確認することですプロビジョニングするワークロードによって必要に応じて、 PVC 定義をさらにカスタマイズできます。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

8. 「 OC 」コマンドを発行して、 PVC を作成します。作成中の元のボリュームのサイズによっては作成にしばらく時間がかかることがあるため、作成が完了した時点でこのプロセスを監視できます。

```
[netapp-user@rhel7 trident-installer]$ oc create -f pvc-basic.yaml
persistentvolumeclaim/basic created

[netapp-user@rhel7 trident-installer]$ oc get pvc
NAME    STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
basic   Bound    pvc-3445b5cc-df24-453d-a1e6-b484e874349d   1Gi
RWO            basic-csi      5s
```

"次：解決策の検証 / ユースケース"

**Copyright Information**

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

**Trademark Information**

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.